EMachine Analysis Notebook

This notebook contains comprehensive analysis of Epsilon Machines (EMachines) for various stochastic processes and quantum state analysis.

Import Libraries and Setup

First, let's import all necessary libraries and set up the environment.

Reloading the emachine module (hot-reload)

- Purpose: Ensure the latest changes to `emachine.py` are used without restarting the kernel.
- What it does: Removes cached `emachine` modules from `sys.modules`, invalidates import caches, and reloads.
- When to use: Run this after editing `emachine.py` to test changes immediately.
- Output: Prints which modules were cleared and confirms successful reload.

```
Found cached emachine modules (1):
  - emachine
Deleted cached emachine entries from sys.modules and invalidated import caches.
Successfully imported 'emachine' -> <module 'emachine' from '/Users/ernestchan/Deskto
p/Msc/emachine.py'>
EM assigned to emachine.EMachine -> <class 'emachine.EMachine'>
```

1. Even Process Analysis

The even process is a fundamental example where we analyze a binary stochastic process with parameter P.

```
Even process with P = 0.4
Transition tensor shape: (2, 2, 2)
EMachine dimension: 2

Observable O:
[[ 1  0]
 [ 0 -1]]
```

Analyze Observable Decay (helper function)

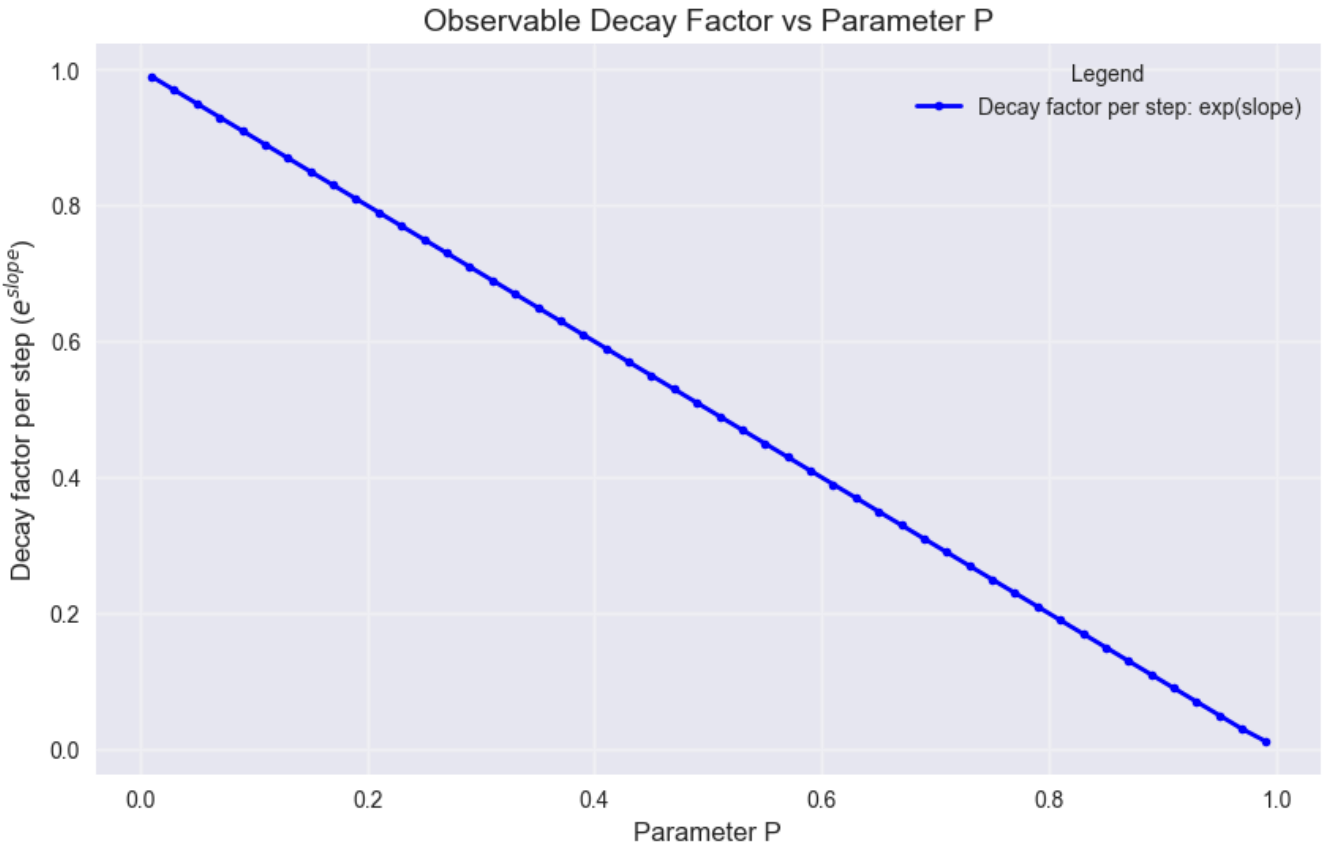
- Purpose: Fit the decay of an observable vs. iteration `n`.
- Inputs: parameter `p`, `max_n` (default 100), and observable `O` from earlier.
- Outputs:
  - Two plots (observable over `n`, and log-absolute with linear fit).

    - Printed slope of the exponential decay where `slope = d/dn [log|observable|]`.
- Notes: Handles non-finite values robustly and shows a fitted guide line if possible.

```
Functions updated to use reloaded EMachine class
```

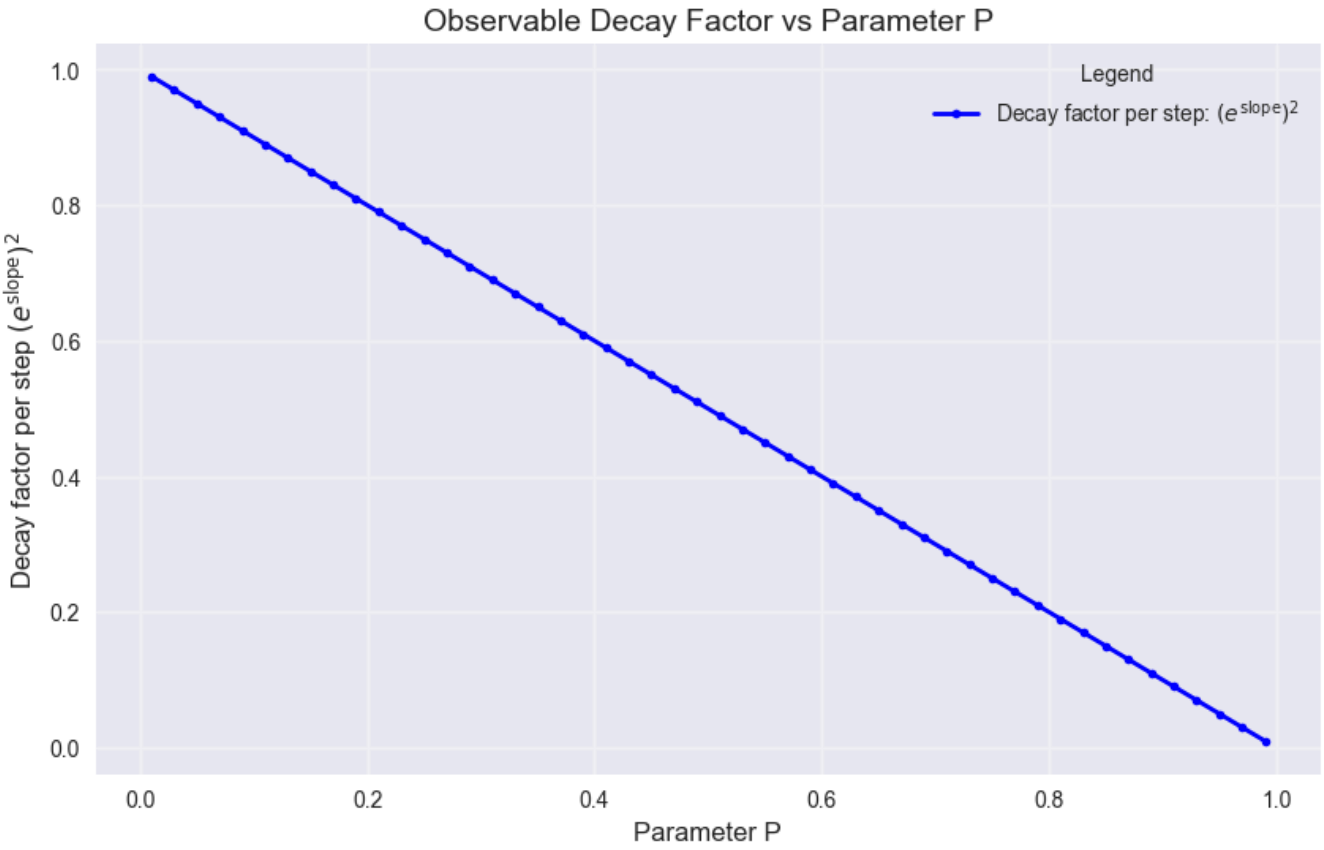
Parameter Sweep (observable O)

- Purpose: Scan `P∈(0,1)` and estimate decay slopes quickly using short `n` range.
- Method: Compute correlation `corr(l)` for `l=1..9`, fit `log|corr|` vs `n`.
- Output: Line plot of decay factor per step `exp(slope)` vs `P`, with the strongest decay annotated.
- Caveat: Short `n` window gives a rough slope; use longer `n` for precision.



Parameter Sweep (observable O2)

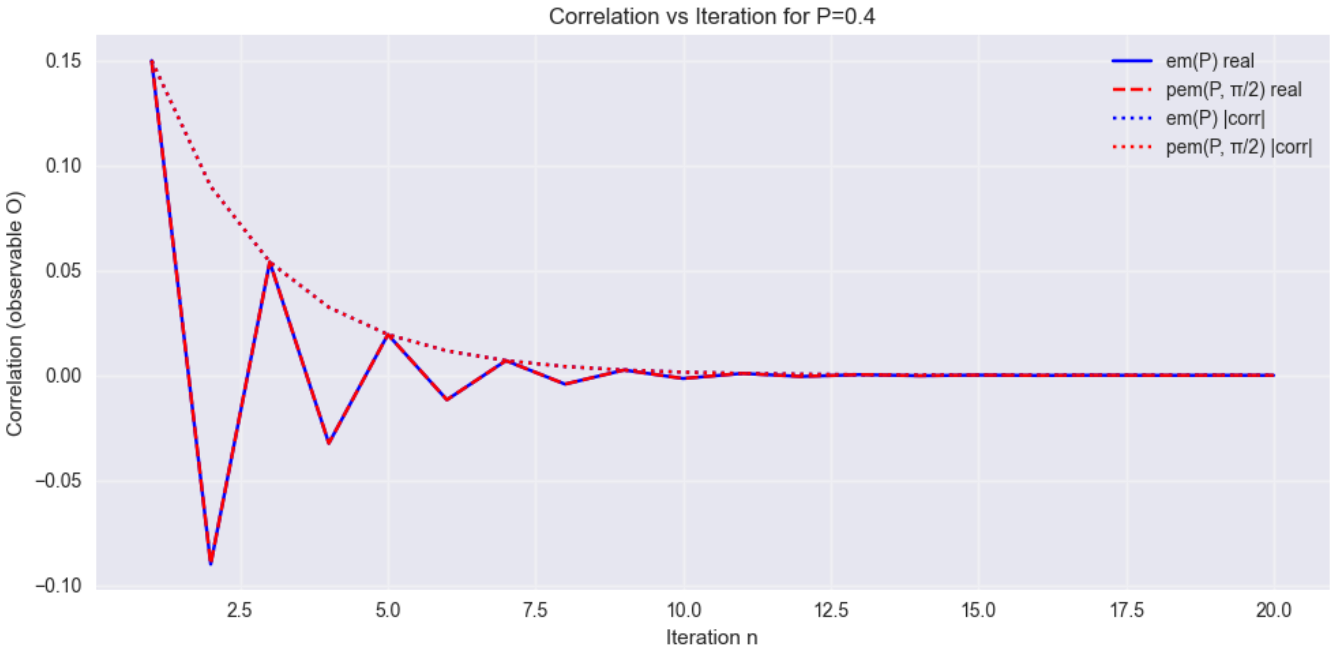
- Purpose: Repeat sweep using `O2 = [[0,1],[1,0]]` to contrast decay vs `O`.
- Output: Plot of `(e^{slope})^2` vs `P` capturing a squared-decay interpretation.
- Tip: Compare positions of minima/maxima between the two sweeps to identify sensitivity to observable choice.



Correlation comparison: base vs phase-modified

The cell below computes the correlation observable for the base even-process EMachine (`em_even(P)`) and the phase-modified EMachine (`pem_even(P, φ)`) at a single representative phase (ϕ = π/2).

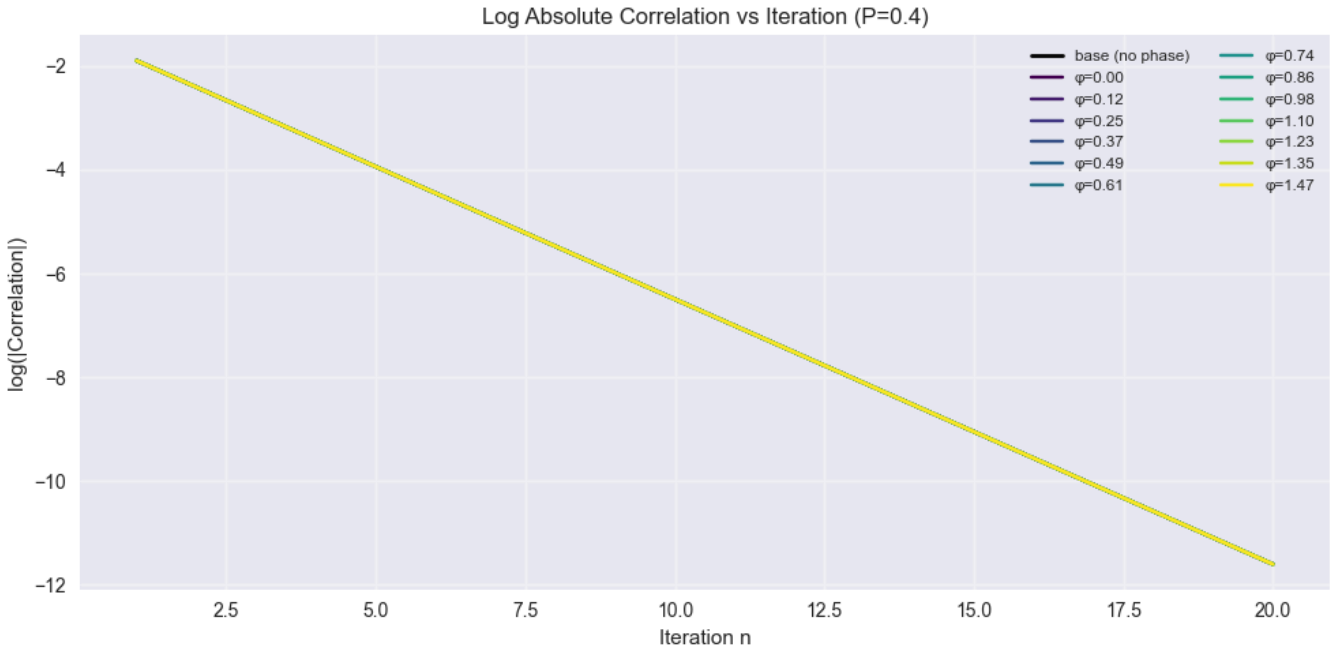
- We compute `corr(n) = em.correlation(0, n)` for n=1..20 for both machines.
- The plots show the real part and the absolute value of the correlation to highlight any phase-induced oscillations and changes in decay behaviour.
- Use these plots to visually compare decay rates and phase effects before running a multi-phase sweep.



Multi-phase sweep: log(|correlation|)

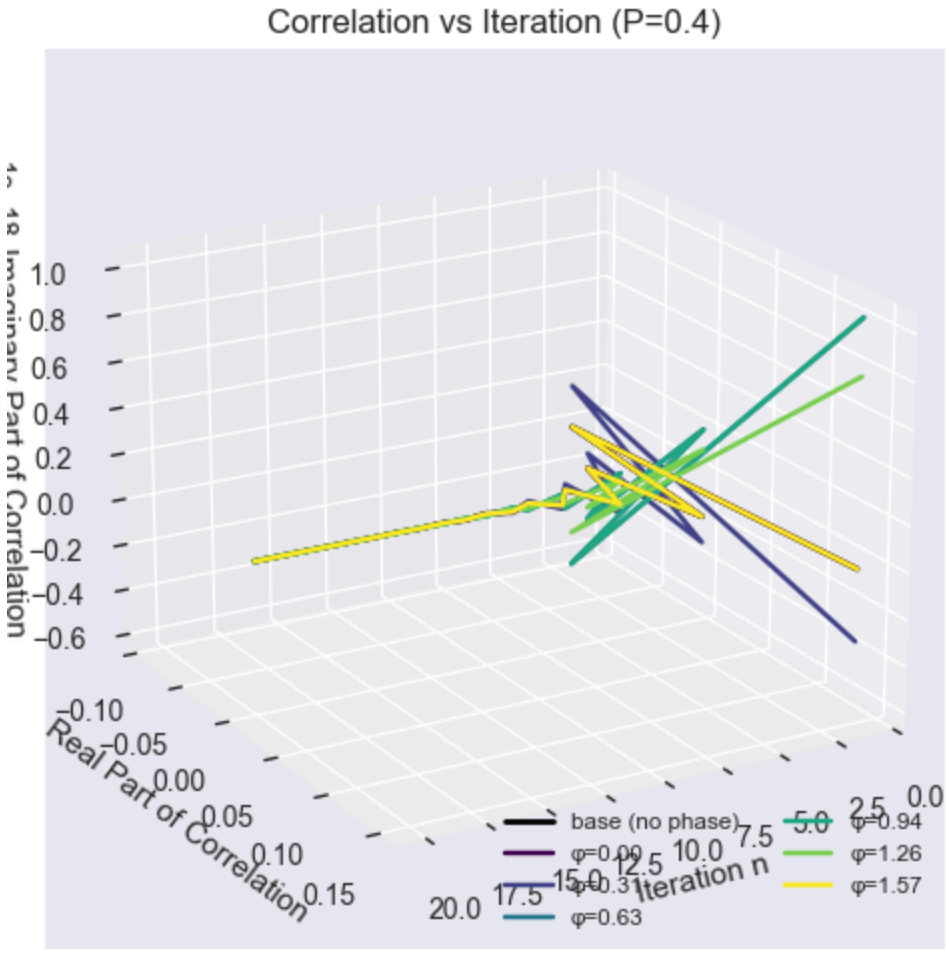
This cell performs a sweep over several phase values ϕ in [0, π/2) and plots log(|corr(n)|) vs iteration n.

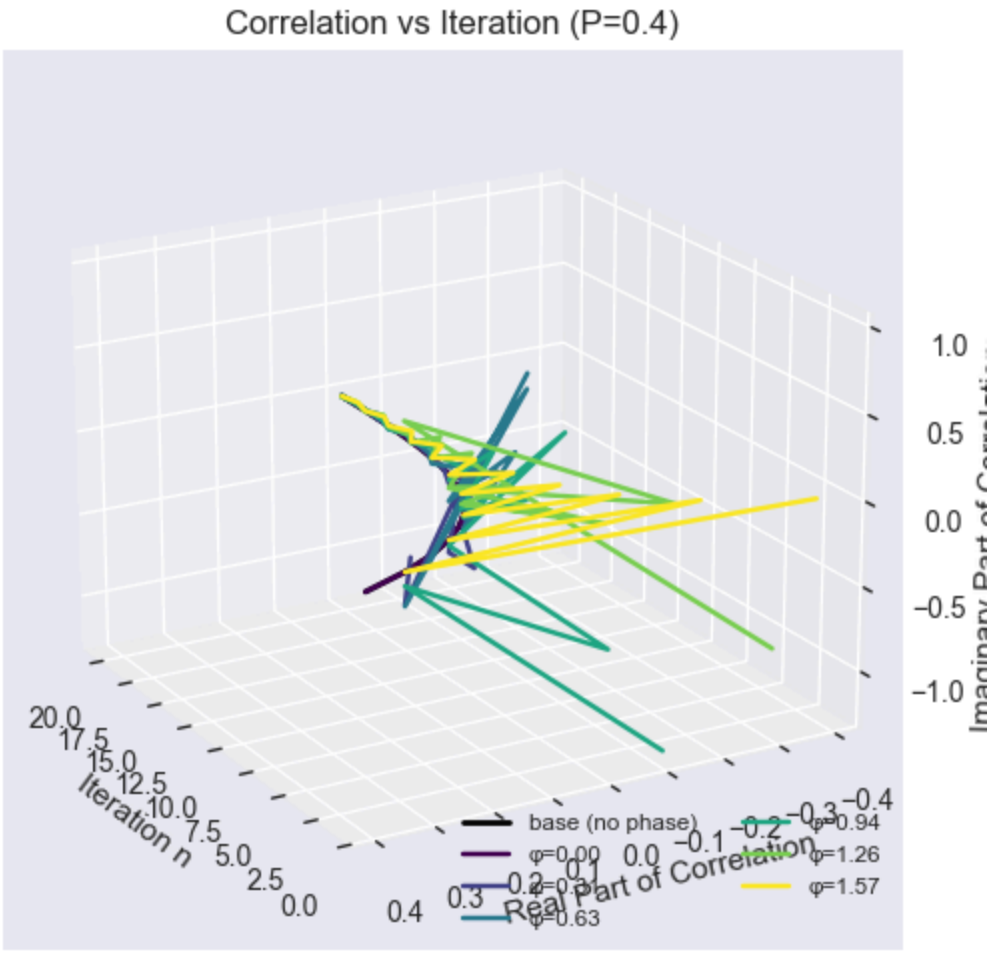
- Each colored line is a different ϕ; compare the slopes and offsets to assess whether phase alters decay rates or only introduces oscillatory components.
- We clip tiny magnitudes to avoid taking the logarithm of zero. Phases that produce numerical issues are skipped.



3D Correlation Trajectories Across Phases

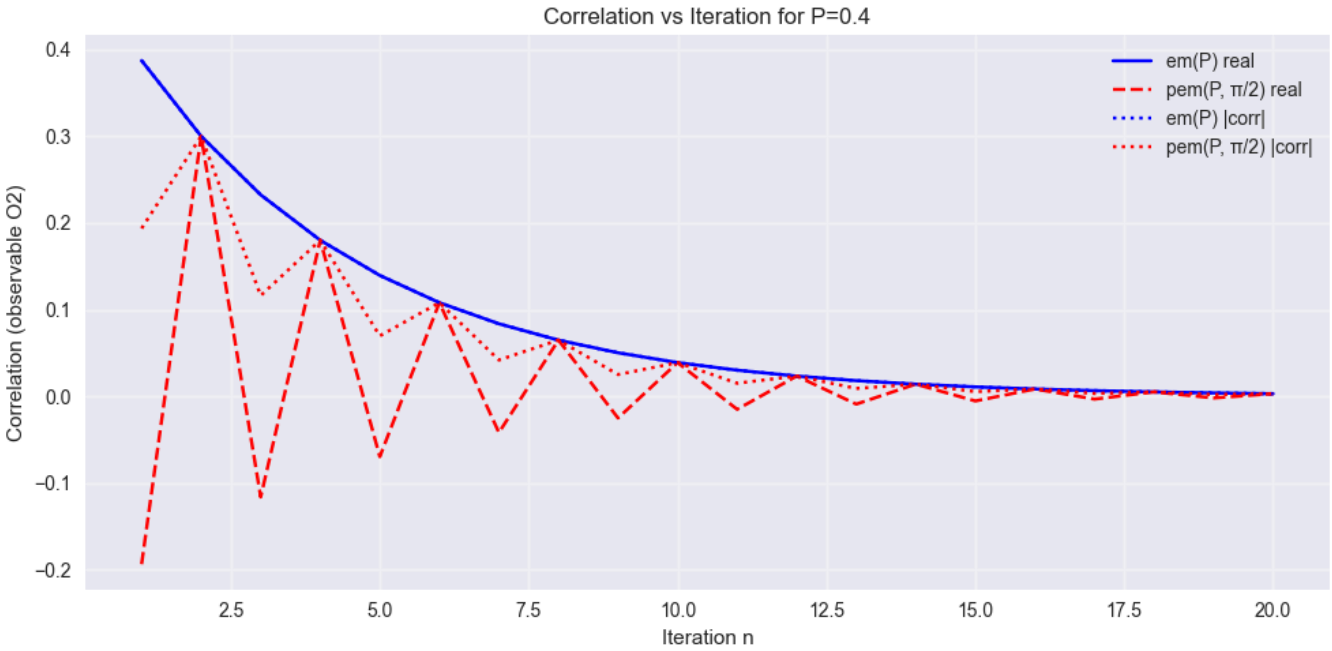
- Purpose: Visualize complex correlation trajectories `Re(corr), Im(corr)` vs `n` for multiple phases.
- Baseline: Includes no-phase curve for reference.
- Output: 3D lines per phase; rotate view to inspect geometry and phase-induced changes.
- Insight: Oscillation patterns and envelope growth/decay become clearer in 3D.





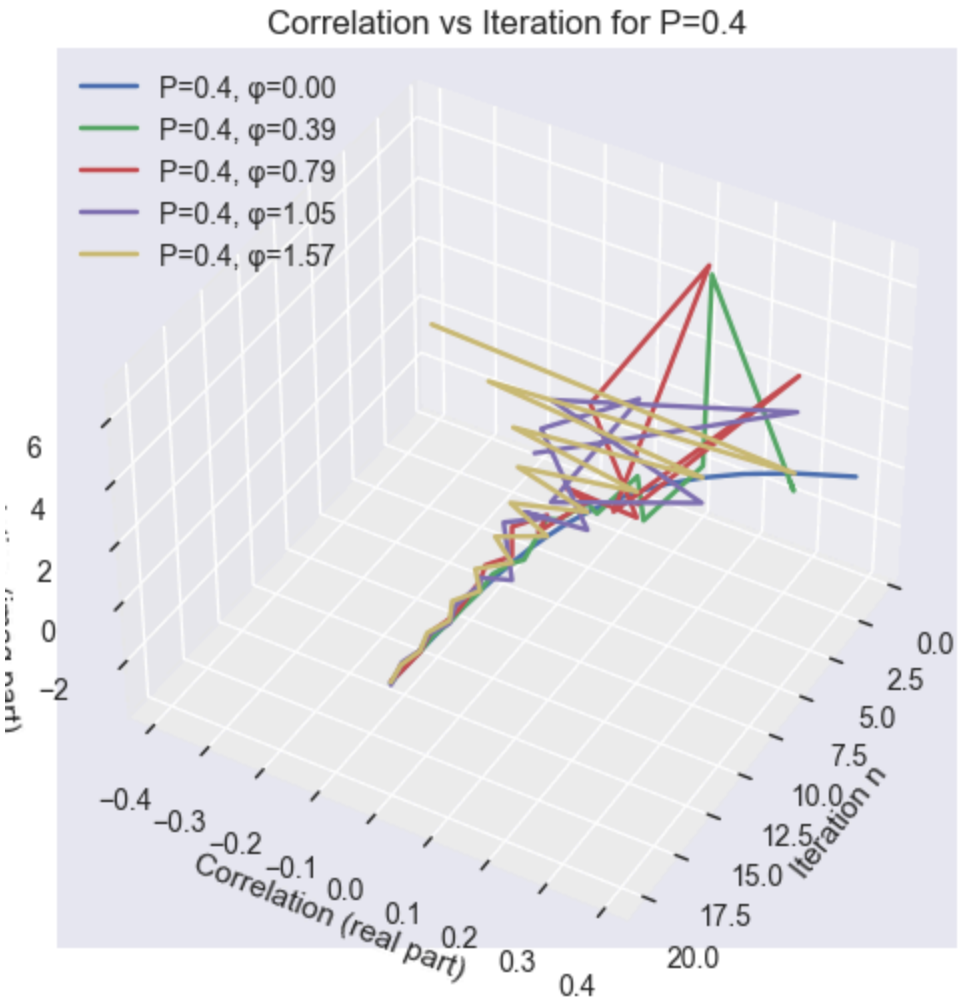
3D Phase Sweep (O observable)

- Purpose: Plot `corr(n)` in 3D (real vs imaginary vs iteration) for multiple phases.
- Output: One line per phase; camera view is preset but can be rotated.
- Use: Compare path geometry to assess phase effects beyond simple magnitude plots.



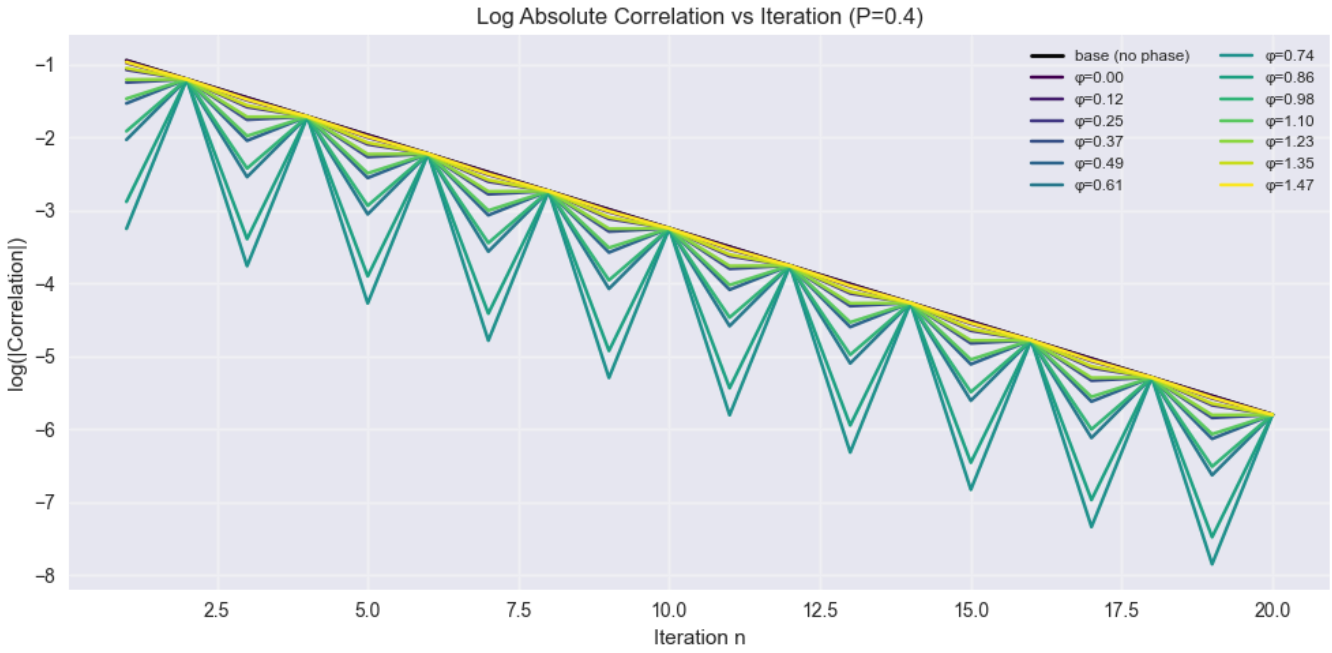
Correlation vs Iteration using O_2 (2D)

- Purpose: Repeat correlation comparison (base vs phase) using alternate observable `02`.
- Plots: Real parts and absolute values vs `n`.
- Insight: Different observables can highlight different dynamical features.



3D Correlation vs Iteration (multiple phases, O_2)

- Purpose: Show full complex trajectory for several phases using `02`.
- View: Elevation/azimuth preset; rotate interactively as needed.
- Legend: Each line labeled with phase value for quick comparison.



Plot correlation function for em(P) and pem(P, pi/2)

Uses existing: T_even_process, em_even, EM, O (observable), np, plt

fallback for P if not defined earlier in the notebook

try: P except NameError: P = 0.4

PHASE = np.pi/2 def pem_even(p, phi): """ Phase-modified even-process EMachine: multiply the second symbol's transition slice by exp(i*phi). """ return PEM(T_even_process(p).astype(complex), np.array([0, phi]))

n = np.arange(1,21)

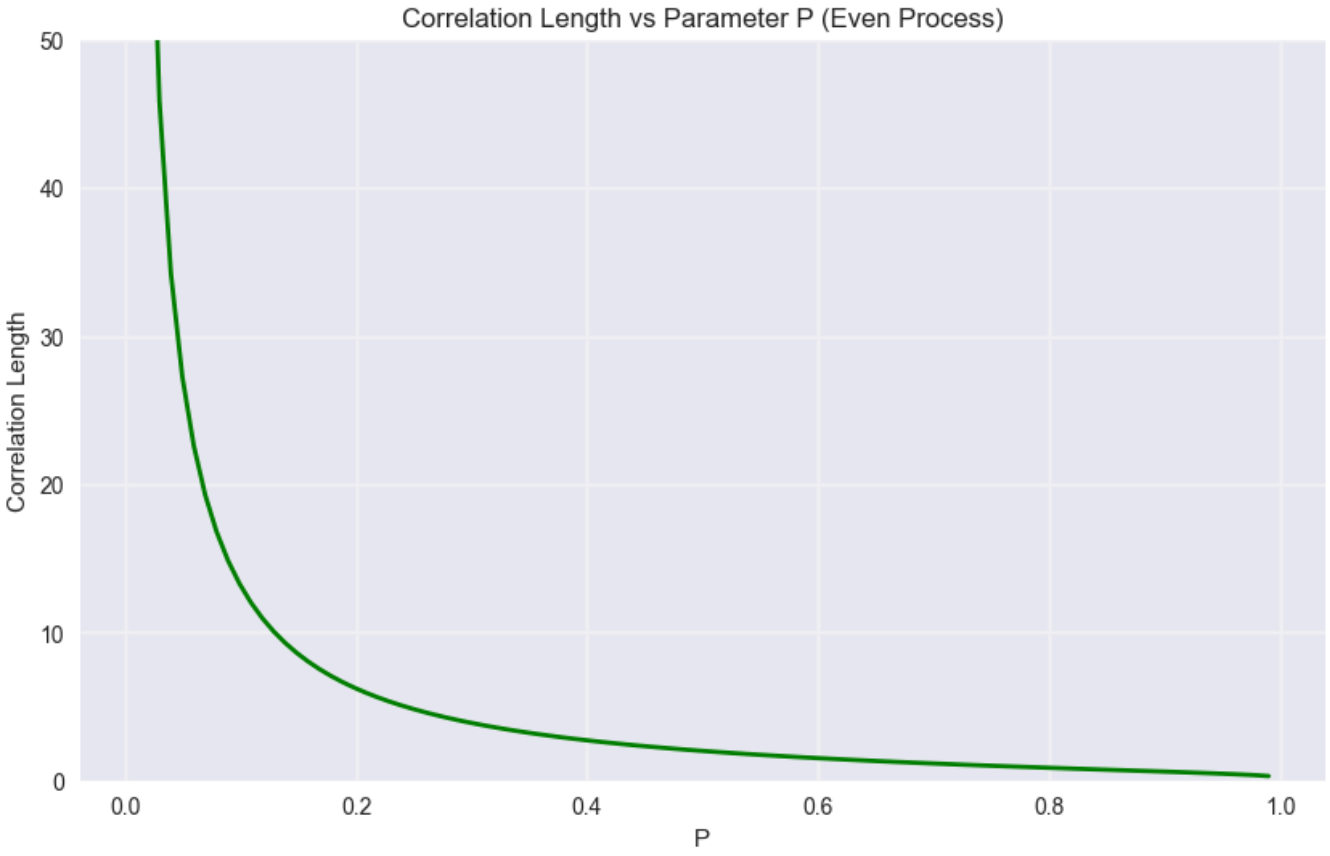
em_base = em_even(P) em_phase = pem_even(P, PHASE) corr_base = np.array([em_base.correlation(O, k) for k in n]) corr_phase = np.array([em_phase.correlation(O, k) for k in n])

plt.figure(figsize=(10, 5)) plt.plot(n, np.real(corr_base), 'b-', label='em(P) real') plt.plot(n, np.real(corr_phase), 'r--', label='pem(P, π/2) real') plt.plot(n, np.abs(corr_base), 'b:', label='em(P) |corr|') plt.plot(n, np.abs(corr_phase), 'r:', label='pem(P, π/2) |corr|')

plt.xlabel('Iteration n') plt.ylabel('Correlation (observable O)') plt.title(f'Correlation vs Iteration for P={P}') plt.grid(True, alpha=0.3) plt.legend() plt.tight_layout() plt.show()

Correlation Length Analysis (Even Process)

- Purpose: Compute and plot correlation length across `P∈(0,1)`.
- Output: 2D curve of correlation length vs `P` with finite-range clipping for clarity.
- Note: Infinite/very large values are handled (clipped/NaN) to keep the plot readable.



2. Upset Gambler Process

The upset gambler process involves two parameters P and Q representing different transition probabilities.

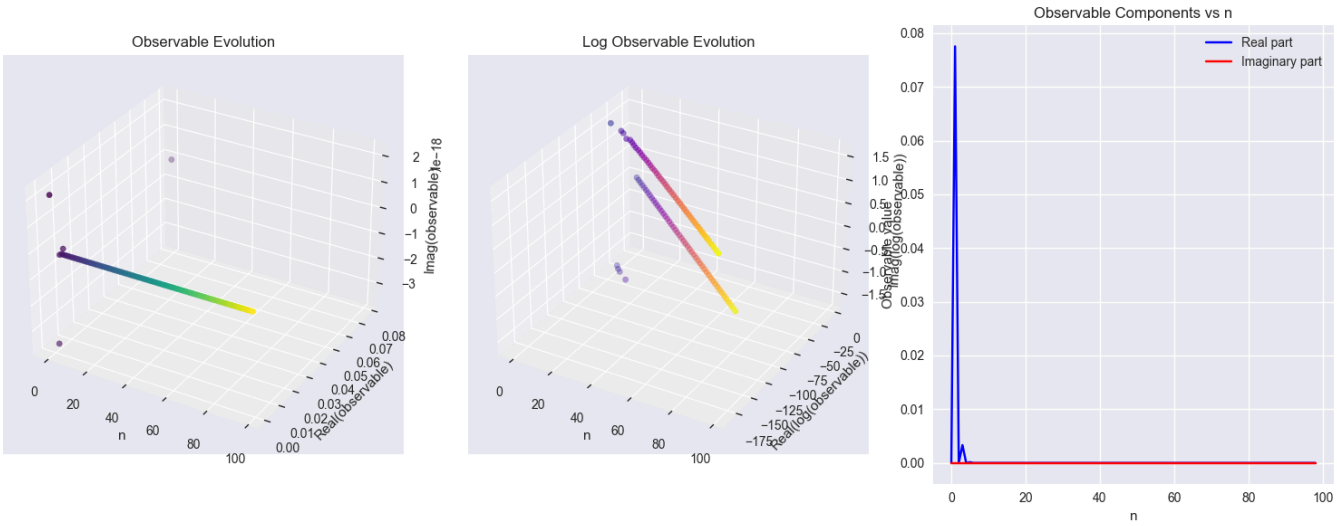
```
Upset gambler with P = 0.8, Q = 0.6
Transition tensor shape: (2, 2, 2)
EMachine dimension: 2

Eigenvalues of E matrix:
[-0.8       +0.j  1.        +0.j  0.69282032+0.j -0.69282032+0.j]
Correlation length: 3.106

Memory measures:
Statistical memory: 0.991+0.000j
Quantum statistical memory: 0.238-0.000j
Topological memory: 1.000
```

Complex Observable Analysis (Upset Gambler)

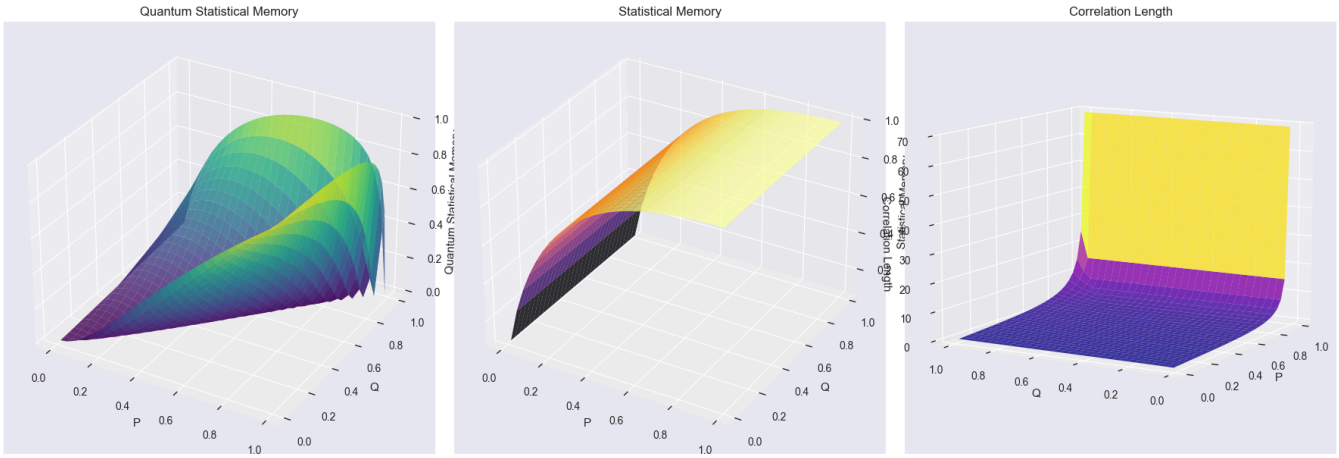
- Purpose: Evaluate real/imag components of a complex observable over `n`.
- Outputs: 3-panel figure (3D evolution, 3D log evolution, 2D components).
- Insight: Visualize oscillations and decay in both raw and log-transformed spaces.



Parameter Space Scan (Upset Gambler: P, Q)

- Purpose: Map memory measures and correlation length over a grid of `(P,Q)`.
- Outputs: Three 3D surfaces for Quantum/Statistical memory and Correlation length.
- Notes: Values are clipped/NaN-handled to improve readability; adjust grid for speed/quality.

```
Computing parameter space analysis...
Progress: 0.0%
Progress: 16.7%
/var/folders/21/h8xsn48s3_nfh_2qh8ddbz5w0000gn/T/ipykernel_71591/1952633179.py:23: Co
mplexWarning: Casting complex values to real discards the imaginary part
  qsm[j, i] = em_temp.quantum_statistical_memory()
/var/folders/21/h8xsn48s3_nfh_2qh8ddbz5w0000gn/T/ipykernel_71591/1952633179.py:24: Co
mplexWarning: Casting complex values to real discards the imaginary part
  sm[j, i] = em_temp.statistical_memory()
Progress: 33.3%
Progress: 50.0%
Progress: 66.7%
Progress: 83.3%
Progress: 66.7%
Progress: 83.3%
```



3. Three-State Systems

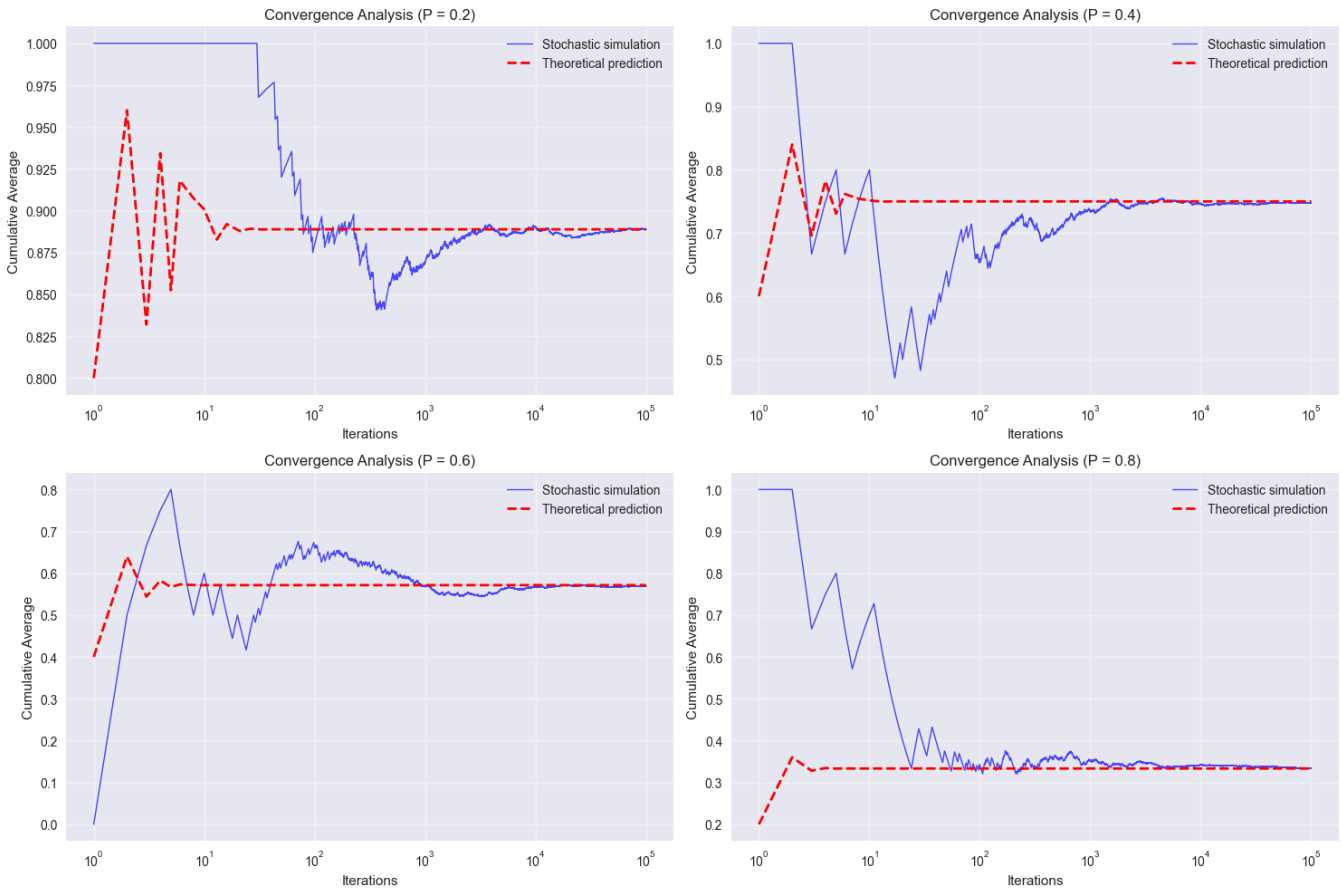
Analysis of more complex three-state stochastic processes.

```
=== Three-State System 1 (Cyclic) ===
Parameter P = 0.4
Lambda matrix diagonal:
[0.80064077 0.42365927 0.42365927]
Statistical memory: 1.585-0.000j

=== Three-State System 2 (Complex) ===
Parameters P = 0.4, Q = 0.8, R = 0.2
Classical eigenvalues:
[1. +0.j         0.3+0.26457513j 0.3-0.26457513j]
Statistical memory: 1.379-0.000j
```

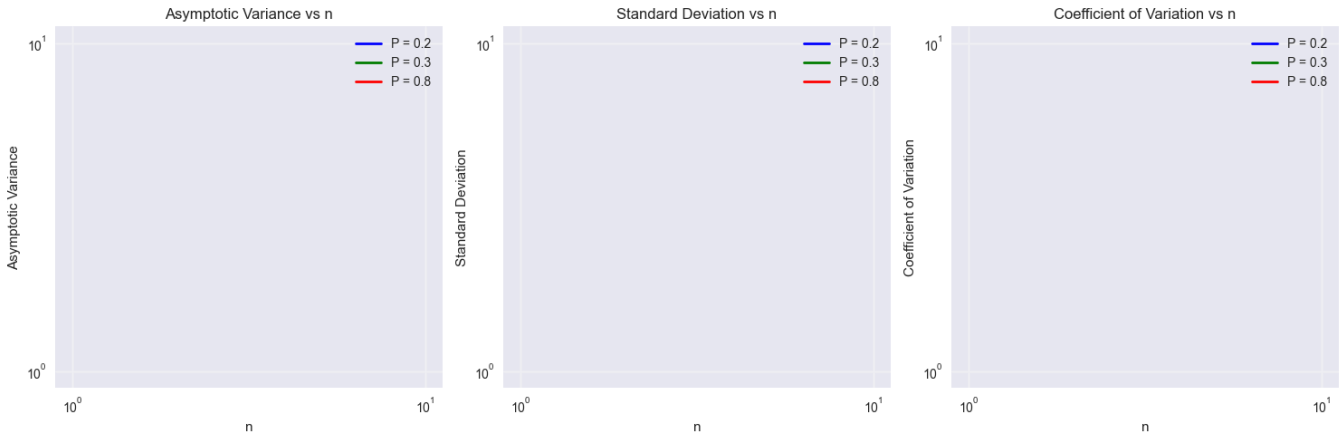
Stochastic Convergence vs Theoretical Prediction

- Purpose: Compare Monte Carlo cumulative averages to theoretical emission predictions.
- Method: Log-spaced sampling points for theory; long-run cumulative average for simulation.
- Output: Semi-log plots for several `P` values with overlays of theory vs simulation.



Variance, Std Dev, Coefficient of Variation

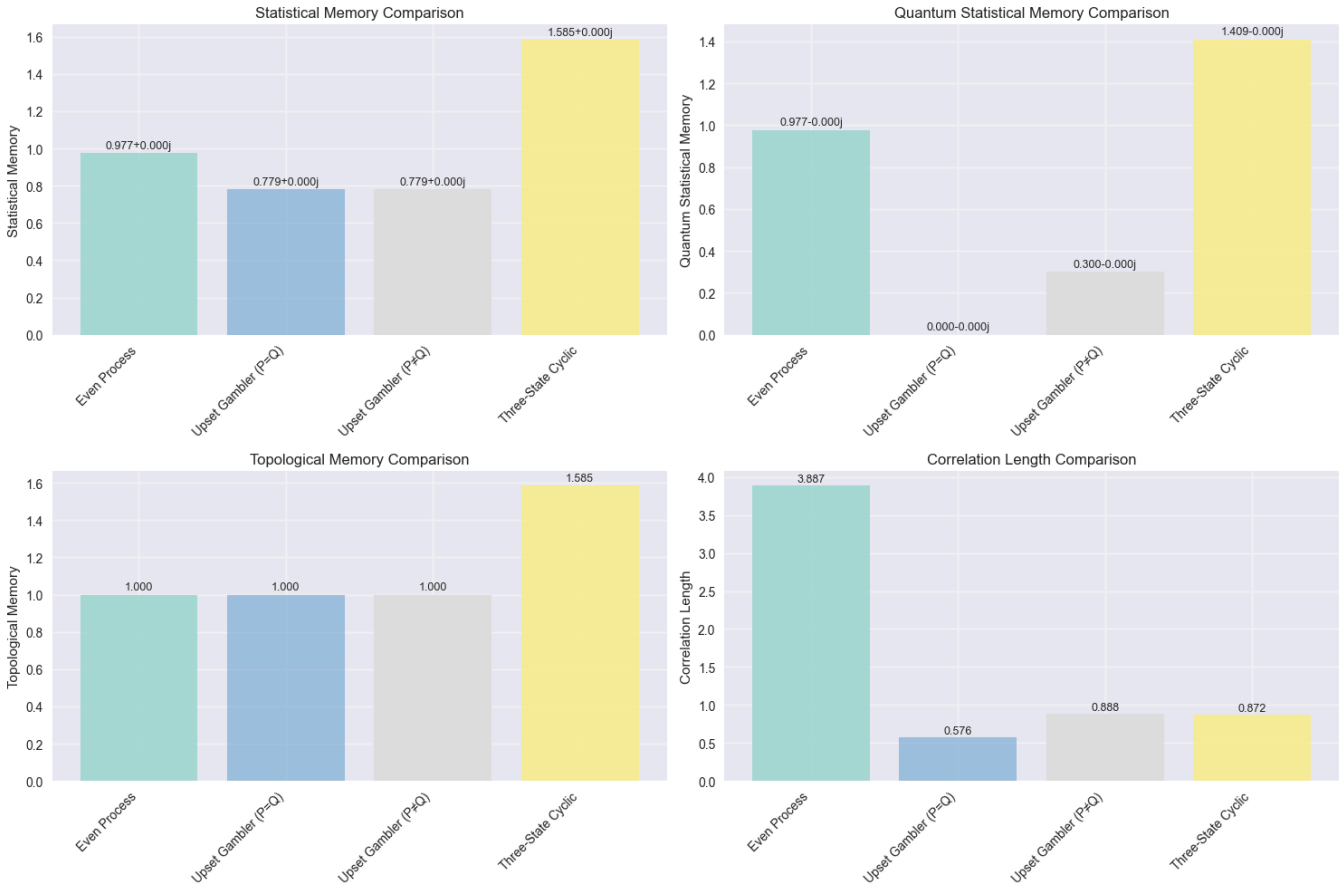
- Purpose: Study scaling of variance-related quantities over `n` for multiple `P`.
- Outputs: Three log-log subplots (variance, std dev, CV) to assess asymptotic behavior.
- Note: Handles potential non-finite values; legends annotate parameter settings.



5. Summary and Comparative Analysis

This section provides summary visualizations comparing different processes and their key properties.

```
/opt/anaconda3/envs/physics_3_11_9/lib/python3.11/site-packages/matplotlib/cbook.py:1
719: ComplexWarning: Casting complex values to real discards the imaginary part
  return math.isfinite(val)
/opt/anaconda3/envs/physics_3_11_9/lib/python3.11/site-packages/matplotlib/transform
s.py:757: ComplexWarning: Casting complex values to real discards the imaginary part
  points = np.asarray(points, float)
/opt/anaconda3/envs/physics_3_11_9/lib/python3.11/site-packages/matplotlib/cbook.py:1
719: ComplexWarning: Casting complex values to real discards the imaginary part
  return math.isfinite(val)
/opt/anaconda3/envs/physics_3_11_9/lib/python3.11/site-packages/matplotlib/transform
s.py:757: ComplexWarning: Casting complex values to real discards the imaginary part
  points = np.asarray(points, float)
/opt/anaconda3/envs/physics_3_11_9/lib/python3.11/site-packages/matplotlib/text.py:90
6: ComplexWarning: Casting complex values to real discards the imaginary part
  y = float(self.convert_yunits(self._y))
/opt/anaconda3/envs/physics_3_11_9/lib/python3.11/site-packages/matplotlib/transform
s.py:757: ComplexWarning: Casting complex values to real discards the imaginary part
  points = np.asarray(points, float)
/opt/anaconda3/envs/physics_3_11_9/lib/python3.11/site-packages/matplotlib/text.py:76
3: ComplexWarning: Casting complex values to real discards the imaginary part
  posy = float(self.convert_yunits(y))
```




```
=== Numerical Comparison ===
Process                   Stat Mem   QStat Mem  Topo Mem   Corr Len
------------------------------------------------------------------------
Even Process              0.977+0.000j 0.977-0.000j 1.000      3.887
Upset Gambler (P=Q)       0.779+0.000j 0.000-0.000j 1.000      0.576
Upset Gambler (P≠Q)       0.779+0.000j 0.300-0.000j 1.000      0.888
Three-State Cyclic        1.585+0.000j 1.409-0.000j 1.585      0.872
```

6. Interactive Analysis Tools

This section provides utility functions for custom analysis of any EMachine process.

```
Example: Custom symmetric process
=== Custom Symmetric Process Analysis ===
Tensor shape: (2, 2, 2)
Dimension: 2
Number of symbols: 2

Eigenvalues of E matrix:
  λ_0: (2.4999999999999982+0j)
  λ_1: (0.08+0j)
  λ_2: (-0.20000000000000034+0j)
  λ_3: (-0.20000000000000007+0j)

Statistical Memory: 2.3721-0.0000j
Quantum Statistical Memory: -0.0000+0.0000j
Topological Memory: 1.0000
Correlation Length: 0.4307

Canonical forms computed successfully
```

Conclusions

This notebook provides a comprehensive analysis framework for Epsilon Machines across different types of stochastic processes:

1. **Even Process**: Simple binary process demonstrating basic EMachine properties
2. **Upset Gambler**: Two-parameter system showing complex parameter space behavior
3. **Three-State Systems**: More complex processes with richer dynamics
4. **Stochastic Simulations**: Validation of theoretical predictions through Monte Carlo methods
5. **Comparative Analysis**: Direct comparison of different process types

Key insights:

- Memory measures vary significantly across process types and parameters
- Observable decay rates provide insights into system dynamics
- Stochastic simulations converge to theoretical predictions
- Parameter space analysis reveals complex relationships in multi-parameter systems

The modular structure allows for easy extension to new process types and analysis methods.